

E-COMMERCE SALES ANALYTICS

SQL Project Presentation

JOINS

WINDOW FUNCTIONS

CTEs

SUBQUERIES

Shadmaan Ahmad

Data Analytics Practice Project · MySQL 8.0+

Project Overview

Objective

Analyze a relational e-commerce dataset spanning customers, orders, products, and order line items in MySQL, in order to understand customer value, product performance, and sales trends. The project is built entirely around four core SQL techniques: joins, window functions, CTEs, and subqueries, applied across 26 progressively harder business questions.

Dataset Snapshot

● Customers	1,000 rows
● Products	64 catalog items
● Orders	300 rows
● Order Line Items	985 rows
● Order Date Range	2019-02-10 → 2020-02-06

Tech Stack



MySQL 8.0+

Relational engine, window functions & CTEs



MySQL Workbench

Query editor, schema design, import wizard



LOAD DATA INFILE

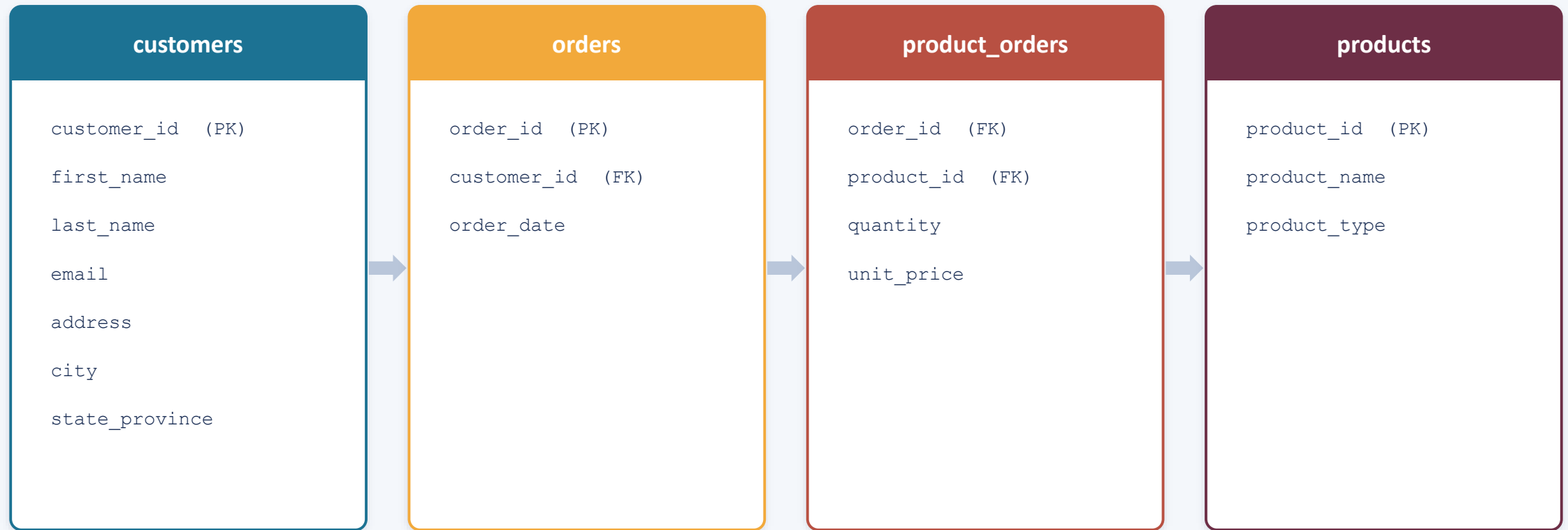
ETL — bulk CSV ingestion with type casting



4 Normalized Tables

customers · orders · product_orders · products

Database Schema



product_orders links orders to products via a composite key (order_id, product_id). Revenue for any line item = quantity × unit_price.

TROUBLESHOOTING

Real-World Data Challenges

The raw CSVs weren't import-ready. Working through these issues end-to-end mirrors what real data engineering looks like.



Character-encoding mismatch

A city name (“Tangwāni”) broke the GUI import wizard, which defaulted to Windows-1252 instead of UTF-8.



local_infile access restriction

MySQL blocked client-side file loads until local_infile was enabled on both the server and the Workbench connection.



Missing trailing newline

The final CSV row had no line terminator, so LOAD DATA INFILE couldn't parse it as a complete row.



Non-ISO date format

Dates arrived as M/D/YYYY text; STR_TO_DATE() with a %c/%e/%Y mask converted them into real DATE values.



Orphaned foreign keys

21 product_ids referenced in product_orders don't exist in the products catalog — loaded with FOREIGN_KEY_CHECKS=0.

SQL Techniques at a Glance



Joins

INNER and LEFT JOINS connect customers, orders, products, and line items — the foundation every later query builds on.



Window Functions

RANK, ROW_NUMBER, LAG, and running SUM() compute rankings, sequences, and trends without collapsing rows.



CTEs

WITH clauses break multi-step logic — segmentation, retention, monthly comparisons — into readable, reusable pieces.



Subqueries

Scalar and correlated subqueries filter rows against averages and rankings computed on the fly.

1 Customer Revenue Analysis — every customer with ≥ 1 order: order count, quantity, and total revenue.

```
SELECT C.CUSTOMER_ID,
CONCAT(C.FIRST_NAME, ' ', C.LAST_NAME) AS CUSTOMER_NAME,
COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
SUM(PO.QUANTITY) AS TOTAL_QUANTITY,
SUM(PO.QUANTITY*PO.UNIT_PRICE) AS TOTAL_REVENUE
FROM CUSTOMERS C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID
JOIN PRODUCT_ORDERS PO ON O.ORDER_ID = PO.ORDER_ID
GROUP BY C.CUSTOMER_ID
ORDER BY TOTAL_REVENUE DESC;
```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 					
	CUSTOMER_ID	CUSTOMER_NAME	TOTAL_ORDERS	TOTAL_QUANTITY	TOTAL_REVENUE
▶	2	Agata Pude	2	78	14887.81
	151	Alfy Epine	2	79	14180.33
	79	Nealy Glide	2	89	13822.68
	157	Hobard Pittaway	1	84	13423.47
	74	Brooke Binden	2	74	12338.14

2 Product Performance — total revenue and quantity sold for every product that has been ordered.

```
SELECT P.PRODUCT_ID,
P.PRODUCT_NAME,
P.PRODUCT_TYPE,
SUM(PO.QUANTITY) AS TOTAL_QUANTITY,
SUM(PO.QUANTITY * PO.UNIT_PRICE) AS TOTAL_REVENUE
FROM PRODUCTS P JOIN PRODUCT_ORDERS PO ON
P.PRODUCT_ID = PO.PRODUCT_ID
GROUP BY PRODUCT_ID
ORDER BY TOTAL_REVENUE DESC;
```

	PRODUCT_ID	PRODUCT_NAME	PRODUCT_TYPE	TOTAL_QUANTITY	TOTAL_REVENUE
▶	38	Toaster and toaster ovens	oven	18	4051.45
	3	Pressure fryer	fryer	20	3731.27
	14	Wet grinder	grinder	23	3667.90
	82	Electric cooker	cooker	18	3445.80
	7	Mess kit	specialty	15	3081.73
	24	Microwave oven	oven	21	2907.11
	37	Beehive oven	oven	12	2877.90
	58	Tabun oven	oven	17	2790.76
	48	Hot box (appliance)	oven	23	2761.77
	54	Vacuum fryer	fryer	11	2714.13
	75	Instant Pot	cooker	13	2557.97
	23	Butane torch	cooker	13	2466.48

3

Product-Type Revenue Analysis — revenue rolled up by product type, with distinct products sold.

```
SELECT P.PRODUCT_TYPE,
COUNT(DISTINCT P.PRODUCT_ID) AS TOTAL_PRODUCT_SOLD,
SUM(PO.QUANTITY) AS TOTAL_QUANTITY,
SUM(PO.QUANTITY * PO.UNIT_PRICE) AS TOTAL_REVENUE
FROM PRODUCTS P JOIN PRODUCT_ORDERS PO
ON P.PRODUCT_ID = PO.PRODUCT_ID
GROUP BY P.PRODUCT_TYPE
ORDER BY TOTAL_REVENUE DESC;
```

Result Grid				
		Filter Rows:	Export:	Wrap Cell Content:
	PRODUCT_TYPE	TOTAL_PRODUCT_SOLD	TOTAL_QUANTITY	TOTAL_REVENUE
▶	oven	19	192	30697.60
	cooker	13	102	15993.86
	specialty	12	83	13025.51
	fryer	5	56	11013.73
	grinder	1	23	3667.90
	heater	2	25	3521.32
	steamer	2	22	3400.56
	coffee	2	15	2791.09
	cooker	1	14	151.84
	specialy	1	11	2306.58
	grill	1	13	1496.56
	roaster	1	4	510.64

4

Customers With No Orders — LEFT JOIN to surface customers who have never ordered.

```
SELECT C.CUSTOMER_ID, CONCAT(FIRST_NAME, ' ', LAST_NAME) AS CUSTOMER_NAME,
CITY, STATE_PROVINCE
FROM CUSTOMERS C LEFT JOIN ORDERS O ON
C.CUSTOMER_ID = O.CUSTOMER_ID
WHERE O.ORDER_ID IS NULL;
```

	CUSTOMER_ID	CUSTOMER_NAME	CITY	STATE_PROVINCE
▶	5	Leighton Spreckley	La Tola	
	11	Kimmie Stallard	Dukou	
	14	Libbey Blackney	Baixo Guandu	
	15	Bonita Berzen	Zelenodolsk	
	16	Gibb Kaplan	Bungtiang Barat	
	17	Mel Dover	Princeville	QC
	18	Jelene Bortolussi	Jianli	
	21	Corilla Van Dale	Ekazhevo	
	25	Kailey Doubleday	Liucheng	
	26	Berkley Wakley	Huatanshan	

5

Products Never Ordered — LEFT JOIN to find catalog products with zero order lines.

```
SELECT P.PRODUCT_ID, P.PRODUCT_NAME, P.PRODUCT_TYPE
FROM PRODUCTS P LEFT JOIN PRODUCT_ORDERS PO
ON P.PRODUCT_ID = PO.PRODUCT_ID
WHERE PO.PRODUCT_ID IS NULL;
```

	PRODUCT_ID	PRODUCT_NAME	PRODUCT_TYPE
▶	22	Slow cooker	cooker
	41	Self-cleaning oven	oven
	60	Barbecue grill	grill
	77	Waffle iron	specialty

6

Order-Level Revenue — total value, item count, and customer name for every order.

```
SELECT O.ORDER_ID, O.ORDER_DATE, O.CUSTOMER_ID,
CONCAT(C.FIRST_NAME, ' ', C.LAST_NAME) AS CUSTOMER_NAME,
SUM(PO.QUANTITY) AS TOTAL_ITEMS,
SUM(PO.QUANTITY * PO.UNIT_PRICE) AS ORDER_VALUE
FROM ORDERS O
JOIN CUSTOMERS C ON O.CUSTOMER_ID = C.CUSTOMER_ID
JOIN PRODUCT_ORDERS PO ON O.ORDER_ID = PO.ORDER_ID
GROUP BY O.ORDER_ID, O.ORDER_DATE, O.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME
ORDER BY O.ORDER_ID;
```

	ORDER_ID	ORDER_DATE	CUSTOMER_ID	CUSTOMER_NAME	TOTAL_ITEMS	ORDER_VALUE
▶	1	2019-03-22	142	Chick Auchterlony	37	5176.25
	2	2019-08-27	12	Orazio Sleaford	43	7061.70
	3	2020-02-01	101	Sheri Stamper	45	9542.97
	4	2019-02-12	35	Davy Forsaith	32	6049.71
	5	2019-07-20	32	Eleanor Brehat	22	3266.83
	6	2020-01-07	166	Clay Campey	41	7541.13
	7	2019-10-16	51	Tye Hosten	19	3598.50
	8	2019-04-16	40	Ferdy Reagan	35	5172.58
	9	2019-09-21	50	Sky Szepe	36	4535.99

7

Rank Customers by Revenue — DENSE_RANK() over each customer's total revenue.

```
WITH CUSTOMER_REVENUE AS (
    SELECT C.CUSTOMER_ID,
    CONCAT(C.FIRST_NAME, ' ', C.LAST_NAME) AS CUSTOMER_NAME,
    SUM(PO.QUANTITY * PO.UNIT_PRICE) AS TOTAL_REVENUE
    FROM CUSTOMERS C
    JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID
    JOIN PRODUCT_ORDERS PO ON O.ORDER_ID = PO.ORDER_ID
    GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME
)
SELECT CUSTOMER_ID, CUSTOMER_NAME, TOTAL_REVENUE,
DENSE_RANK() OVER (ORDER BY TOTAL_REVENUE DESC) AS REVENUE_RANK
FROM CUSTOMER_REVENUE
ORDER BY REVENUE_RANK;
```

	CUSTOMER_ID	CUSTOMER_NAME	TOTAL_REVENUE	REVENUE_RANK
▶	2	Agata Pude	14887.81	1
	151	Alfy Epine	14180.33	2
	79	Nealy Glide	13822.68	3
	157	Hobard Pittaway	13423.47	4
	74	Brooke Binden	12338.14	5
	76	Ferdinand Ferrini	11949.25	6
	22	Marcella Fiennes	11878.67	7
	73	Florri Howey	10631.95	8
	7	Dewain Eblein	10601.98	9

8

Top 3 Products per Type — ROW_NUMBER() partitioned by product_type, ordered by revenue.

```
WITH PRODUCT_REVENUE AS (
    SELECT P.PRODUCT_ID, P.PRODUCT_NAME, P.PRODUCT_TYPE,
    SUM(PO.QUANTITY * PO.UNIT_PRICE) AS TOTAL_REVENUE
    FROM PRODUCTS P
    JOIN PRODUCT_ORDERS PO ON P.PRODUCT_ID = PO.PRODUCT_ID
    GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME, P.PRODUCT_TYPE
),
RANKED_PRODUCTS AS (
    SELECT *,
    ROW_NUMBER() OVER (PARTITION BY PRODUCT_TYPE ORDER BY TOTAL_REVENUE DESC) AS RN
    FROM PRODUCT_REVENUE
)
SELECT PRODUCT_ID, PRODUCT_NAME, PRODUCT_TYPE, TOTAL_REVENUE, RN
FROM RANKED_PRODUCTS
WHERE RN <= 3
ORDER BY PRODUCT_TYPE, RN;
```

	PRODUCT_ID	PRODUCT_NAME	PRODUCT_TYPE	TOTAL_REVENUE	RN
▶	52	Fire pot	cooker	2410.84	1
	85	Coffee pot	coffee	1812.93	1
	84	Espresso machine	coffee	978.16	2
	82	Electric cooker	cooker	3445.80	1
	75	Instant Pot	cooker	2557.97	2
	23	Butane torch	cooker	2466.48	3
	3	Pressure fryer	fryer	3731.27	1
	54	Vacuum fryer	fryer	2714.13	2
	26	Deep fryer	fryer	1994.30	3

9

Running Monthly Revenue — monthly revenue plus a cumulative SUM() over time.

```
WITH MONTHLY_REVENUE AS (
    SELECT DATE_FORMAT(O.ORDER_DATE, '%Y-%m') AS MONTH,
           SUM(PO.QUANTITY * PO.UNIT_PRICE) AS MONTHLY_REVENUE
    FROM ORDERS O
    JOIN PRODUCT_ORDERS PO ON O.ORDER_ID = PO.ORDER_ID
    GROUP BY DATE_FORMAT(O.ORDER_DATE, '%Y-%m')
)
SELECT MONTH, MONTHLY_REVENUE,
       SUM(MONTHLY_REVENUE) OVER (ORDER BY MONTH) AS CUMULATIVE_REVENUE
FROM MONTHLY_REVENUE
ORDER BY MONTH;
```

	MONTH	MONTHLY_REVENUE	CUMULATIVE_REVENUE
▶	2019-02	25453.94	25453.94
	2019-03	28740.03	54193.97
	2019-04	39411.69	93605.66
	2019-05	70221.95	163827.61
	2019-06	43163.45	206991.06
	2019-07	37316.42	244307.48
	2019-08	14990.32	259297.80
	2019-09	33883.53	293181.33
	2019-10	24670.73	317852.06
	2019-11	52525.40	370377.46
	2019-12	46259.79	416637.25
	2020-01	19568.23	436205.48

10

Customer Order Sequence — ROW_NUMBER() per customer, ordered by order date.

```
SELECT O.CUSTOMER_ID, O.ORDER_ID, O.ORDER_DATE,
       ROW_NUMBER() OVER (PARTITION BY O.CUSTOMER_ID
                          ORDER BY O.ORDER_DATE, O.ORDER_ID) AS ORDER_SEQUENCE
FROM ORDERS O
ORDER BY O.CUSTOMER_ID, ORDER_SEQUENCE;
```

	CUSTOMER_ID	ORDER_ID	ORDER_DATE	ORDER_SEQUENCE
▶	1	242	2019-02-25	1
	1	87	2020-01-23	2
	2	27	2019-04-24	1
	2	70	2019-10-05	2
	3	168	2019-02-22	1
	3	196	2019-04-07	2
	3	271	2019-07-11	3
	3	164	2019-07-14	4
	4	277	2019-07-24	1
	6	252	2019-06-15	1
	7	51	2019-07-01	1
	7	42	2019-09-26	2

11

Days Between Customer Orders — LAG() to compute the gap since the previous order.

```

SELECT CUSTOMER_ID, ORDER_ID, ORDER_DATE,
DATEDIFF(ORDER_DATE, LAG(ORDER_DATE) OVER (PARTITION BY CUSTOMER_ID
ORDER BY ORDER_DATE)) AS DAYS_SINCE_PREVIOUS_ORDER
FROM ORDERS
ORDER BY CUSTOMER_ID, ORDER_DATE;

```

	CUSTOMER_ID	ORDER_ID	ORDER_DATE	DAYS_SINCE_PREVIOUS_ORDER
▶	1	242	2019-02-25	NULL
	1	87	2020-01-23	332
	2	27	2019-04-24	NULL
	2	70	2019-10-05	164
	3	168	2019-02-22	NULL
	3	196	2019-04-07	44
	3	271	2019-07-11	95
	3	164	2019-07-14	3
	4	277	2019-07-24	NULL
	6	252	2019-06-15	NULL
	7	51	2019-07-01	NULL
	7	42	2019-09-26	87

12

Order Contribution to Revenue — each order's % share of its customer's total revenue.

```

WITH ORDER_VALUES AS (
  SELECT O.ORDER_ID, O.CUSTOMER_ID,
    SUM(PO.QUANTITY * PO.UNIT_PRICE) AS ORDER_VALUE
  FROM ORDERS O
  JOIN PRODUCT_ORDERS PO ON O.ORDER_ID = PO.ORDER_ID
  GROUP BY O.ORDER_ID, O.CUSTOMER_ID
)
SELECT ORDER_ID, CUSTOMER_ID, ORDER_VALUE,
SUM(ORDER_VALUE) OVER (PARTITION BY CUSTOMER_ID) AS CUSTOMER_TOTAL_REVENUE,
ROUND(ORDER_VALUE / SUM(ORDER_VALUE) OVER (PARTITION BY CUSTOMER_ID) * 100, 2) AS PCT_OF_CUSTOMER_REVENUE
FROM ORDER_VALUES
ORDER BY CUSTOMER_ID, ORDER_VALUE DESC;

```

	ORDER_ID	CUSTOMER_ID	ORDER_VALUE	CUSTOMER_TOTAL_REVENUE	PCT_OF_CUSTOMER_REVENUE
▶	70	2	8705.66	14887.81	58.48
	27	2	6182.15	14887.81	41.52
	42	7	6231.67	10601.98	58.78
	51	7	4370.31	10601.98	41.22
	19	8	5116.32	7291.87	7291.87
	60	8	2175.55	7291.87	29.84
	18	9	9987.58	9987.58	100.00
	2	12	7061.70	7061.70	100.00
	35	13	4023.41	4023.41	100.00
	40	22	6649.50	11878.67	55.98
	58	22	5229.17	11878.67	44.02
	44	23	3727.92	3727.92	100.00

13

Highest-Value Order per Customer — ROW_NUMBER(), tie-broken by the earlier order.

```

WITH ORDER_VALUES AS (
    SELECT O.ORDER_ID, O.CUSTOMER_ID, O.ORDER_DATE,
    SUM(PO.QUANTITY * PO.UNIT_PRICE) AS ORDER_VALUE
    FROM ORDERS O
    JOIN PRODUCT_ORDERS PO ON O.ORDER_ID = PO.ORDER_ID
    GROUP BY O.ORDER_ID, O.CUSTOMER_ID, O.ORDER_DATE
),
RANKED AS (
    SELECT *,
    ROW_NUMBER() OVER (PARTITION BY CUSTOMER_ID ORDER BY ORDER_VALUE DESC, ORDER_DATE ASC) AS RN
    FROM ORDER_VALUES
)
SELECT ORDER_ID, CUSTOMER_ID, ORDER_DATE, ORDER_VALUE
FROM RANKED
WHERE RN = 1
ORDER BY CUSTOMER_ID;

```

	ORDER_ID	CUSTOMER_ID	ORDER_DATE	ORDER_VALUE
▶	70	2	2019-10-05	8705.66
	42	7	2019-09-26	6231.67
	19	8	2019-05-26	5116.32
	18	9	2019-11-12	9987.58
	2	12	2019-08-27	7061.70
	35	13	2019-04-04	4023.41
	40	22	2019-07-19	6649.50
	44	23	2019-12-21	3727.92
	63	27	2019-03-31	3338.30
	20	29	2019-06-20	2666.21
	17	30	2019-11-15	2811.47
	5	32	2019-07-20	3266.83

14

Monthly Revenue Growth — LAG() to compare each month against the previous month.

```

WITH MONTHLY_REVENUE AS (
    SELECT DATE_FORMAT(O.ORDER_DATE, '%Y-%m') AS MONTH,
    SUM(PO.QUANTITY * PO.UNIT_PRICE) AS MONTHLY_REVENUE
    FROM ORDERS O
    JOIN PRODUCT_ORDERS PO ON O.ORDER_ID = PO.ORDER_ID
    GROUP BY DATE_FORMAT(O.ORDER_DATE, '%Y-%m')
)
SELECT MONTH, MONTHLY_REVENUE,
LAG(MONTHLY_REVENUE) OVER (ORDER BY MONTH) AS PREVIOUS_MONTH_REVENUE,
MONTHLY_REVENUE - LAG(MONTHLY_REVENUE) OVER (ORDER BY MONTH) AS REVENUE_CHANGE,
ROUND(
    (MONTHLY_REVENUE - LAG(MONTHLY_REVENUE) OVER (ORDER BY MONTH))
    / LAG(MONTHLY_REVENUE) OVER (ORDER BY MONTH) * 100, 2
) AS GROWTH_PERCENTAGE
FROM MONTHLY_REVENUE
ORDER BY MONTH;

```

	MONTH	MONTHLY_REVENUE	PREVIOUS_MONTH_REVENUE	REVENUE_CHANGE	GROWTH_PERCENTAGE
▶	2019-02	25453.94	NULL	NULL	NULL
	2019-03	28740.03	25453.94	3286.09	12.91
	2019-04	39411.69	28740.03	10671.66	37.13
	2019-05	70221.95	39411.69	30810.26	30810.26
	2019-06	43163.45	70221.95	-27058.50	-38.53
	2019-07	37316.42	43163.45	-5847.03	-13.55
	2019-08	14990.32	37316.42	-22326.10	-59.83
	2019-09	33883.53	14990.32	18893.21	126.04
	2019-10	24670.73	33883.53	-9212.80	-27.19
	2019-11	52525.40	24670.73	27854.67	112.91
	2019-12	46259.79	52525.40	-6265.61	-11.93
	2020-01	19568.23	46259.79	-26691.56	-57.70

15

Customer Purchase Segmentation — CTE classifies customers as High / Medium / Low value.

```
WITH CUSTOMER_SUMMARY AS (
    SELECT C.CUSTOMER_ID,
    CONCAT(C.FIRST_NAME, ' ', C.LAST_NAME) AS CUSTOMER_NAME,
    COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
    SUM(PO.QUANTITY * PO.UNIT_PRICE) AS TOTAL_REVENUE
    FROM CUSTOMERS C
    JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID
    JOIN PRODUCT_ORDERS PO ON O.ORDER_ID = PO.ORDER_ID
    GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME
)
SELECT CUSTOMER_ID, CUSTOMER_NAME, TOTAL_ORDERS, TOTAL_REVENUE,
CASE
    WHEN TOTAL_REVENUE >= 1000 THEN 'High Value'
    WHEN TOTAL_REVENUE >= 500 THEN 'Medium Value'
    ELSE 'Low Value'
END AS SEGMENT
FROM CUSTOMER_SUMMARY
ORDER BY TOTAL_REVENUE DESC;
```

	CUSTOMER_ID	CUSTOMER_NAME	TOTAL_ORDERS	TOTAL_REVENUE	SEGMENT
▶	2	Agata Pude	2	14887.81	High Value
	151	Alfy Epine	2	14180.33	High Value
	79	Nealy Glide	2	13822.68	High Value
	157	Hobard Pittaway	1	13423.47	High Value
	74	Brooke Binden	2	12338.14	High Value
	76	Ferdinand Ferrini	2	11949.25	High Value
	22	Marcella Fiennes	2	11878.67	High Value
	73	Florri Howey	2	10631.95	High Value
	7	Dewain Eblein	2	10601.98	High Value
	50	Sky Szepe	2	10180.70	High Value
	9	Graehme Kilvington	1	9987.5 9987.58	High Value
	121	Carver Simoneton	2	9951.20	High Value

16

Product Type Performance — CTE rollup with products sold, quantity, revenue, and averages.

```
WITH PRODUCT_REVENUE AS (
    SELECT P.PRODUCT_ID, P.PRODUCT_TYPE,
    SUM(PO.QUANTITY) AS TOTAL_QUANTITY,
    SUM(PO.QUANTITY * PO.UNIT_PRICE) AS TOTAL_REVENUE
    FROM PRODUCTS P
    JOIN PRODUCT_ORDERS PO ON P.PRODUCT_ID = PO.PRODUCT_ID
    GROUP BY P.PRODUCT_ID, P.PRODUCT_TYPE
)
SELECT PRODUCT_TYPE,
COUNT(PRODUCT_ID) AS PRODUCTS_SOLD,
SUM(TOTAL_QUANTITY) AS TOTAL_QUANTITY,
SUM(TOTAL_REVENUE) AS TOTAL_REVENUE,
ROUND(AVG(TOTAL_REVENUE), 2) AS AVERAGE_PRODUCT_REVENUE
FROM PRODUCT_REVENUE
GROUP BY PRODUCT_TYPE
ORDER BY TOTAL_REVENUE DESC;
```

	PRODUCT_TYPE	PRODUCTS_SOLD	TOTAL_QUANTITY	TOTAL_REVENUE	AVERAGE_PRODUCT_REVENUE
▶	oven	19	192	30697.60	1615.66
	cooker	13	102	15993.86	1230.30
	specialty	12	83	13025.51	1085.46
	fryer	5	56	11013.73	2202.75
	grinder	1	23	3667.90	3667.90
	heater	2	25	3521.32	1760.66
	steamer	2	22	3400.56	1700.28
	coffee	2	15	2791.09	1395.55
	cooker	1	14	2410.84	2410.84
	specialty	1	11	2306.58	2306.58
	grill	1	13	1496.56	1496.56
	roaster	1	4	510.64	510.64

```

WITH MONTHLY_CUSTOMER_REVENUE AS (
    SELECT DATE_FORMAT(O.ORDER_DATE, '%Y-%m') AS MONTH,
           O.CUSTOMER_ID,
           SUM(PO.QUANTITY * PO.UNIT_PRICE) AS REVENUE
    FROM ORDERS O
    JOIN PRODUCT_ORDERS PO ON O.ORDER_ID = PO.ORDER_ID
    GROUP BY DATE_FORMAT(O.ORDER_DATE, '%Y-%m'), O.CUSTOMER_ID
),
RANKED AS (
    SELECT *,
           ROW_NUMBER() OVER (PARTITION BY MONTH ORDER BY REVENUE DESC) AS RN
    FROM MONTHLY_CUSTOMER_REVENUE
)
SELECT MONTH, CUSTOMER_ID, REVENUE, RN
FROM RANKED
WHERE RN <= 5
ORDER BY MONTH, RN;

```

	MONTH	CUSTOMER_ID	REVENUE	RN
▶	2019-02	183	6923.94	1
	2019-02	114	6510.81	2
	2019-02	35	6049.71	3
	2019-02	182	5969.48	4
	2019-03	142	8481.11	1
	2019-03	91	6237.46	2
	2019-03	49	6126.28	3
	2019-03	200	4556.88	4
	2019-03	27	3338.30	5
	2019-04	178	9457.93	1
	2019-04	2	6182.15	2
	2019-04	51	5247.49	3

```

WITH ORDER_VALUES AS (
    SELECT O.ORDER_ID, O.CUSTOMER_ID, O.ORDER_DATE,
           SUM(PO.QUANTITY * PO.UNIT_PRICE) AS ORDER_VALUE
    FROM ORDERS O
    JOIN PRODUCT_ORDERS PO ON O.ORDER_ID = PO.ORDER_ID
    GROUP BY O.ORDER_ID, O.CUSTOMER_ID, O.ORDER_DATE
),
FIRST_LAST AS (
    SELECT CUSTOMER_ID,
           FIRST_VALUE(ORDER_VALUE) OVER (PARTITION BY CUSTOMER_ID ORDER BY ORDER_DATE, ORDER_ID) AS FIRST_ORDER_VALUE,
           FIRST_VALUE(ORDER_VALUE) OVER (PARTITION BY CUSTOMER_ID ORDER BY ORDER_DATE DESC, ORDER_ID DESC) AS LAST_ORDER_VALUE
    FROM ORDER_VALUES
)

SELECT DISTINCT CUSTOMER_ID, FIRST_ORDER_VALUE, LAST_ORDER_VALUE,
               (LAST_ORDER_VALUE - FIRST_ORDER_VALUE) AS INCREASE_AMOUNT
FROM FIRST_LAST
WHERE LAST_ORDER_VALUE > FIRST_ORDER_VALUE
ORDER BY INCREASE_AMOUNT DESC;

```

	CUSTOMER_ID	FIRST_ORDER_VALUE	LAST_ORDER_VALUE	INCREASE_AMOUNT
▶	76	2951.19	8998.06	6046.87
	121	3697.05	6254.15	2557.10
	2	6182.15	8705.66	2523.51
	142	3304.86	5176.25	1871.39
	7	4370.31	6231.67	1861.36
	22	5229.17	6649.50	1420.33
	79	6230.21	7592.47	1362.26
	96	4239.07	5568.52	1329.45
	50	4535.99	5644.71	1108.72
	74	6115.45	6222.69	107.24

```

WITH ORDER_VALUES AS (
    SELECT O.ORDER_ID, O.CUSTOMER_ID, O.ORDER_DATE,
    DATE_FORMAT(O.ORDER_DATE, '%Y-%m') AS MONTH,
    SUM(PO.QUANTITY * PO.UNIT_PRICE) AS ORDER_VALUE
    FROM ORDERS O
    JOIN PRODUCT_ORDERS PO ON O.ORDER_ID = PO.ORDER_ID
    GROUP BY O.ORDER_ID, O.CUSTOMER_ID, O.ORDER_DATE
),
MONTHLY_AVG AS (
    SELECT MONTH, AVG(ORDER_VALUE) AS AVG_ORDER_VALUE
    FROM ORDER_VALUES
    GROUP BY MONTH
)
SELECT OV.ORDER_ID, OV.CUSTOMER_ID, OV.ORDER_DATE, OV.ORDER_VALUE, MA.AVG_ORDER_VALUE
FROM ORDER_VALUES OV
JOIN MONTHLY_AVG MA ON OV.MONTH = MA.MONTH
WHERE OV.ORDER_VALUE > MA.AVG_ORDER_VALUE
ORDER BY OV.ORDER_DATE;

```

	ORDER_ID	CUSTOMER_ID	ORDER_DATE	ORDER_VALUE	AVG_ORDER_VALUE
▶	38	114	2019-02-11	6510.81	6363.485000
	85	183	2019-02-12	6923.94	6363.485000
	1	142	2019-03-22	5176.25	4790.005000
	57	49	2019-03-24	6126.28	4790.005000
	78	91	2019-03-29	6237.46	4790.005000
	76	178	2019-04-03	9457.93	5630.241429
	27	2	2019-04-24	6182.15	5630.241429
	73	181	2019-05-02	6487.45	6383.813636
	56	151	2019-05-04	9128.04	6383.813636
	52	117	2019-05-06	8795.26	6383.813636
	37	39	2019-05-11	6832.64	6383.813636
	65	42	2019-05-22	7321.29	6383.813636

```

WITH CUSTOMER_ORDERS AS (
    SELECT CUSTOMER_ID,
    MIN(ORDER_DATE) AS FIRST_ORDER_DATE,
    MAX(ORDER_DATE) AS LAST_ORDER_DATE,
    COUNT(*) AS TOTAL_ORDERS
    FROM ORDERS
    GROUP BY CUSTOMER_ID
)
SELECT CUSTOMER_ID, FIRST_ORDER_DATE, LAST_ORDER_DATE, TOTAL_ORDERS,
DATEDIFF(LAST_ORDER_DATE, FIRST_ORDER_DATE) AS DAYS_BETWEEN_FIRST_AND_LAST_ORDER
FROM CUSTOMER_ORDERS
WHERE TOTAL_ORDERS >= 2
ORDER BY TOTAL_ORDERS DESC;

```

	CUSTOMER_ID	FIRST_ORDER_DATE	LAST_ORDER_DATE	TOTAL_ORDERS	DAYS_BETWEEN_FIRST_AND_LAST_ORDER
▶	84	2019-06-03	2019-07-22	5	49
	39	2019-02-19	2020-01-25	5	340
	192	2019-04-08	2020-01-11	5	278
	50	2019-09-21	2019-12-31	4	101
	3	2019-02-22	2019-07-14	4	142
	76	2019-03-15	2019-12-31	4	291
	72	2019-07-02	2019-10-04	4	94
	12	2019-03-16	2019-09-25	4	193
	74	2019-03-24	2020-01-25	4	307
	22	2019-07-16	2019-12-27	4	164
	93	2019-03-13	2019-10-12	4	213
	128	2019-05-07	2019-09-01	4	117

21

Customers Spending Above Average — HAVING vs. the average revenue across all customers.

```

SELECT C.CUSTOMER_ID,
CONCAT(C.FIRST_NAME, ' ', C.LAST_NAME) AS CUSTOMER_NAME,
SUM(PO.QUANTITY * PO.UNIT_PRICE) AS TOTAL_REVENUE
FROM CUSTOMERS C
JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID
JOIN PRODUCT_ORDERS PO ON O.ORDER_ID = PO.ORDER_ID
GROUP BY C.CUSTOMER_ID, C.FIRST_NAME, C.LAST_NAME
HAVING SUM(PO.QUANTITY * PO.UNIT_PRICE) > (
    SELECT AVG(CUST_REV.REV)
    FROM (
        SELECT SUM(PO2.QUANTITY * PO2.UNIT_PRICE) AS REV
        FROM ORDERS O2
        JOIN PRODUCT_ORDERS PO2 ON O2.ORDER_ID = PO2.ORDER_ID
        GROUP BY O2.CUSTOMER_ID
    ) AS CUST_REV
)
ORDER BY TOTAL_REVENUE DESC;

```

	CUSTOMER_ID	CUSTOMER_NAME	TOTAL_REVENUE
▶	2	Agata Pude	14887.81
	151	Alfy Epine	14180.33
	79	Nealy Glide	13822.68
	157	Hobard Pittaway	13423.47
	74	Brooke Binden	12338.14
	76	Ferdinand Ferrini	11949.25
	22	Marcella Fiennes	11878.67
	73	Florri Howey	10631.95
	7	Dewain Eblein	10601.98
	50	Sky Szepe	10180.70
	9	Graehme Kilvington	9987.58
	121	Carver Simoneton	9951.20

22

Products Above Average Revenue — HAVING vs. the average revenue across all products.

```

SELECT P.PRODUCT_ID, P.PRODUCT_NAME, P.PRODUCT_TYPE,
SUM(PO.QUANTITY * PO.UNIT_PRICE) AS TOTAL_REVENUE
FROM PRODUCTS P
JOIN PRODUCT_ORDERS PO ON P.PRODUCT_ID = PO.PRODUCT_ID
GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME, P.PRODUCT_TYPE
HAVING SUM(PO.QUANTITY * PO.UNIT_PRICE) > (
    SELECT AVG(PROD_REV.REV)
    FROM (
        SELECT SUM(PO2.QUANTITY * PO2.UNIT_PRICE) AS REV
        FROM PRODUCT_ORDERS PO2
        GROUP BY PO2.PRODUCT_ID
    ) AS PROD_REV
)
ORDER BY TOTAL_REVENUE DESC;

```

	PRODUCT_ID	PRODUCT_NAME	PRODUCT_TYPE	TOTAL_REVENUE
▶	38	Toaster and toaster ovens	oven	4051.45
	3	Pressure fryer	fryer	3731.27
	14	Wet grinder	grinder	3667.90
	82	Electric cooker	cooker	3445.80
	7	Mess kit	specialty	3081.73
	24	Microwave oven	oven	2907.11
	37	Beehive oven	oven	2877.90
	58	Tabun oven	oven	2790.76
	48	Hot box (appliance)	oven	2761.77
	54	Vacuum fryer	fryer	2714.13
	75	Instant Pot	cooker	2557.97
	23	Butane torch	cooker	2466.48

```

SELECT O.ORDER_ID, O.CUSTOMER_ID, O.ORDER_DATE,
SUM(PO.QUANTITY * PO.UNIT_PRICE) AS ORDER_VALUE,
(
    SELECT AVG(OV.VALUE)
    FROM (
        SELECT SUM(PO2.QUANTITY * PO2.UNIT_PRICE) AS VALUE
        FROM ORDERS O2
        JOIN PRODUCT_ORDERS PO2 ON O2.ORDER_ID = PO2.ORDER_ID
        WHERE O2.CUSTOMER_ID = O.CUSTOMER_ID
        GROUP BY O2.ORDER_ID
    ) AS OV
) AS CUSTOMER_AVERAGE_ORDER_VALUE
FROM ORDERS O
JOIN PRODUCT_ORDERS PO ON O.ORDER_ID = PO.ORDER_ID
GROUP BY O.ORDER_ID, O.CUSTOMER_ID, O.ORDER_DATE
HAVING SUM(PO.QUANTITY * PO.UNIT_PRICE) > (
    SELECT AVG(OV.VALUE)
    FROM (
        SELECT SUM(PO2.QUANTITY * PO2.UNIT_PRICE) AS VALUE
        FROM ORDERS O2
        JOIN PRODUCT_ORDERS PO2 ON O2.ORDER_ID = PO2.ORDER_ID
        WHERE O2.CUSTOMER_ID = O.CUSTOMER_ID
        GROUP BY O2.ORDER_ID
    ) AS OV
)
ORDER BY O.CUSTOMER_ID, O.ORDER_DATE;

```

	ORDER_ID	CUSTOMER_ID	ORDER_DATE	ORDER_VALUE	CUSTOMER_AVERAGE_ORDER_VALUE
▶	70	2	2019-10-05	8705.66	7443.905000
	42	7	2019-09-26	6231.67	5300.990000
	19	8	2019-05-26	5116.32	3645.935000
	40	22	2019-07-19	6649.50	5939.335000
	64	50	2019-12-31	5644.71	5090.350000
	12	51	2019-04-30	5247.49	4422.995000
	13	73	2019-07-27	6259.21	5315.975000
	24	74	2020-01-25	6222.69	6169.070000
	80	76	2019-12-31	8998.06	5974.625000
	11	79	2019-12-18	7592.47	6911.340000
	62	96	2019-08-21	5568.52	4903.795000
	59	121	2020-02-01	6254.15	4975.600000
	1	142	2019-03-22	5176.25	4240.555000
	56	151	2019-05-04	9128.04	7090.165000

24

Customers With ≥3 Orders — subquery identifies qualifying customer IDs.

```

SELECT C.CUSTOMER_ID,
CONCAT(C.FIRST_NAME, ' ', C.LAST_NAME) AS CUSTOMER_NAME,
C.CITY, C.STATE_PROVINCE, OC.ORDER_COUNT
FROM CUSTOMERS C
JOIN (
    SELECT CUSTOMER_ID, COUNT(*) AS ORDER_COUNT
    FROM ORDERS
    GROUP BY CUSTOMER_ID
    HAVING COUNT(*) >= 3
) AS OC ON C.CUSTOMER_ID = OC.CUSTOMER_ID
ORDER BY OC.ORDER_COUNT DESC;

```

	CUSTOMER_ID	CUSTOMER_NAME	CITY	STATE_PROVINCE	ORDER_COUNT
▶	39	Brigg Fitter	Clénega		5
	84	Arin Semrad	Hang Dong		5
	192	Gallagher Wynne	Jiexiu		5
	3	Reinald MacLucais	Margarita		4
	12	Orazio Sleaford	Agía Triáda		4
	22	Marcella Fiennes	Zhangye		4
	74	Brooke Binden	Santiago Puringla		4
	50	Sky Szepe	Höshööt		4
	76	Ferdinand Ferrini	Markušica		4
	93	Arlan MacCurtain	Yanji		4
	72	Friedrich Schulze	Maguan		4
	128	Zollie Girdler	Rusné		4
	166	Clav Camdev	Ravānsar		4

25

Customers Who Bought a Specific Type — IN subquery for the 'oven' product type.

```

SELECT DISTINCT C.CUSTOMER_ID,
CONCAT(C.FIRST_NAME, ' ', C.LAST_NAME) AS CUSTOMER_NAME
FROM CUSTOMERS C
WHERE C.CUSTOMER_ID IN (
    SELECT O.CUSTOMER_ID
    FROM ORDERS O
    JOIN PRODUCT_ORDERS PO ON O.ORDER_ID = PO.ORDER_ID
    JOIN PRODUCTS P ON PO.PRODUCT_ID = P.PRODUCT_ID
    WHERE P.PRODUCT_TYPE = 'oven'
);

```

	CUSTOMER_ID	CUSTOMER_NAME
▶	35	Davy Forsaith
	69	Ezra Hullett
	49	Malorie Lavielle
	27	Anselm Stockell
	32	Eleanor Brehat
	63	Brianne Tomich
	50	Sky Szepe
	114	Gretal Varnes
	62	Stavros Perfitt
	101	Sheri Stamper
	37	Christiana Truder
	157	Hobard Pittaway
	65	Rodolph Kemet

ANALYSIS

Key Insights

Five takeaways from the business-question results.

1

Insight: 44 customers have placed at least 3 orders, providing a focused repeat-customer segment.

2

Insight: 112 customers have purchased at least one oven-type product — the catalog's clear demand leader.

3

Insight: oven is the top-performing product type, generating 30,697.60 in revenue, nearly double cooker, the next-highest category.

4

Insight: 124 customers qualify as “High Value” spenders under the $\geq 1,000$ total-revenue segmentation.

5

Insight: 847 of the 1,000 customers have never placed an order — a large, untapped segment worth targeted outreach.

RESULTS

Key Findings

Results calculated from the uploaded CSV dataset and the 26 SQL questions.

● Product Analysis

Top revenue product line

oven — 119,216.64 revenue

Top revenue product

Bread machine — 9,987.58

Lowest revenue product type

grinder — 3,887.98

● Sales & Revenue

Highest-revenue month

2019-05 — 57,462.69

Best month-over-month growth

2019-05 — +74.68%

Total revenue (all orders)

465,299.45

● Customer Analysis

Top customer by revenue

Arin Semrad — 8,850.69

High-value customer count

124

Customers never ordering

847

● Data Quality Notes

Orphaned product_ids

21 (in product_orders, missing from products)

Products never ordered

0

Repeat-purchase customers

87

Thank You

E-Commerce Sales Analytics — SQL Project

Shadmaan Ahmad